

Chapter Two

Deep Learning

Basic Principles, Architecture, and Evolutionary Drivers

(Deep Learning: Fundamentals, Architecture, and Evolutionary Drivers)

Shaon Khan

BSc in IIT, JU

Table of Contents

Contents

Table of Contents	2
1. Chapter Summary and Basic Definitions	4
1.1 What is Deep Learning? — A Formal Definition	4
1.2 Representation Learning vs Manual Feature Engineering	4
2. Mathematical and Structural Foundation of Neural Networks	5
2.1 The Perceptron.....	5
2.2 Anatomy of Layers	6
<i>Input Layer</i>	6
<i>Hidden Layer</i>	6
<i>Output Layer</i>	6
2.3 Mathematical Operation and Learning Process (Learning Mechanics)	6
<i>Forward Propagation</i>	6
<i>Loss Function</i>	6
<i>Backpropagation</i>	6
<i>Gradient Descent</i>	7
2.4 Theoretical Benchmark: Universal Approximation Theorem	7
3. Paradigm Comparison and Contrast Analysis	7
3.1 Traditional Machine Learning vs Deep Learning	7
3.2 Statistical Paradigm vs Logical/Neural Paradigm	8
3.3 Hardware Accelerators: CPU vs GPU vs TPU vs NPU	8
4. The Ecosystem of Modern Deep Learning Architectures	9
4.1 ANN / MLP (Artificial Neural Network / Multi-Layer Perceptron)	9
4.2 CNN (Convolutional Neural Networks)	9
4.3 RNN / LSTM (Recurrent Neural Networks)	10
4.4 Transformers.....	10
4.5 GAN (Generative Adversarial Networks).....	10
4.6 Diffusion Models	10
5. The Revival of Deep Learning in the 2010s: Five Driving Forces	11
5.1 The Dataset Revolution	11
5.2 Hardware and Parallel Computing	11
5.3 Rise of Software Frameworks and Libraries	11
5.4 Algorithmic and Architectural Breakthrough Innovations	12

5.5 Open Science and Global Community (Open Science)	12
6. Special Tips and Insights	13
7. Chapter Review and Assessment.....	13
7.1 Summary	13
7.2 Key Mathematical Formulas	14
7.3 Top 5 Interview Questions and Formal Answers	14
<i>Question 1: What is representation learning, and how is it different from manual feature engineering?</i>	14
<i>Question 2: What is the vanishing gradient problem, and how did ReLU/ResNet solve it?</i>	14
<i>Question 3: Compare the architectures of CPU, GPU, TPU, and NPU in the context of tensor operations.</i>	14
<i>Question 4: Why are gradient boosted trees, such as XGBoost, often preferred over deep neural networks for tabular data?</i>	15
<i>Question 5: How does the Transformer's self-attention mechanism overcome the limitations of RNNs?</i>	15
7.4 Multiple Choice Questions (MCQs)	15
7.5 Analytical Questions and Practical Problems	16
<i>Problem 1</i>	16
<i>Problem 2</i>	16

1. Chapter Summary and Basic Definitions

1.1 What is Deep Learning? — A Formal Definition

Deep Learning is a special branch of machine learning. In deep learning, we use artificial neural networks that have many layers. These networks automatically learn abstract and layered representations from data. In the book "Deep Learning" (MIT Press), written by Ian Goodfellow, Yoshua Bengio, and Aaron Courville, deep learning is described as a method of representation learning. In this method, complex ideas are built from simpler ideas, and this building process happens through many processing layers. Each layer takes the output of the layer before it as its input, and creates a representation with a higher level of abstraction.

In 2015, a well-known paper titled "Deep Learning" was published in the journal Nature by Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. This paper explains that deep learning methods are a class of representation-learning methods, where many layers of non-linear modules are stacked one after another. This allows the system to create higher and more abstract features directly from raw data — each layer turns the representation from the layer below into a slightly more abstract representation. In this same paper, the authors also show that stacking enough of these transformation layers makes it possible to learn very complex functions.

Notice that in the term "Deep Learning," the word "Deep" refers to the depth of the network — that is, the number of hidden layers between the input layer and the output layer. As a general rule, a network with more than one hidden layer is called a "deep" neural network, while a network with only one hidden layer is considered a "shallow" network.

1.2 Representation Learning vs Manual Feature Engineering

In traditional machine learning methods, to make a model work well, domain experts had to do manual feature engineering by hand. This means that humans had to decide which features from the raw data were important for the model. This process takes a lot of time, requires a lot of manual effort, and depends heavily on domain-specific knowledge.

Deep learning goes beyond this limitation through a technique called representation learning, or feature learning. In this technique, the model itself automatically discovers the features it needs from raw data. This is mainly based on hierarchical representation, or layered representation — where each layer identifies features that are more abstract and more complex than the layer before it.

For example, in an image processing task such as face recognition, the layered learning process of a Convolutional Neural Network (CNN) works like this:

- First layer: identifies simple edges and corners from raw pixel values.
- Second layer: combines edges to form geometric shapes, textures, and patterns.

- Third layer: combines shapes to identify parts such as eyes, nose, and lips.
- Final layer: combines these parts to build a high-level representation of the whole face or object, and classifies it.

This process of gradually increasing abstraction is what makes deep learning an advanced form of "hierarchical feature learning." Here, without any human help, the model is able to decide on its own which features matter the most. Technical documentation from Google Cloud and IBM also names this feature as the most important thing that sets deep learning apart — a feature that has helped bring a revolution to fields such as computer vision, speech recognition, and natural language processing (NLP).

2. Mathematical and Structural Foundation of Neural Networks

2.1 The Perceptron

The historical starting point of the modern neural network goes back to 1958, when psychologist Frank Rosenblatt proposed a mathematical model called the Perceptron. This was the simplest artificial neural unit, inspired by how a biological neuron works in the human brain, and it was used to solve binary classification problems.

We can understand the mathematical working of a perceptron in three simple steps:

- 1. Input Vector and Weights:** each input feature $x_1, x_2, \dots, x_n$ is multiplied by a matching weight $w_1, w_2, \dots, w_n$. This weight shows how important that input is.
- 2. Weighted Sum and Bias:** we add a bias value (b) to the sum of all these products, and this gives us a single number, z .
- 3. Activation Function:** we pass the value z through an activation function f to get the final output a . This step adds non-linearity to the network.

$$z = \sum_{i=1}^n w_i x_i + b$$

$$a = f(z)$$

Without an activation function, a neural network would only ever be a linear regression model, no matter how many layers we add. That is why we add non-linear functions such as Sigmoid, Tanh, and ReLU. These give the network the power to learn complex, non-linear decision boundaries.

2.2 Anatomy of Layers

Input Layer

The input layer is the first layer of the network. It receives raw data in the form of a numeric vector. It does not do any mathematical computation — it simply acts as a pass-through, sending the data on to the next layer. The number of neurons in this layer depends directly on the number of input features.

Hidden Layer

The layers that sit between the input layer and the output layer are called hidden layers, because their computations and outputs are not directly visible to anyone outside the model — they carry out the network's internal process of abstraction. The "depth" of a network is decided by how many hidden layers it has, and the number of neurons in each layer decides the network's "width." As depth increases, the network can gradually learn more abstract features. As width increases, each layer can hold more patterns at the same time.

Output Layer

The output layer gives the network's final prediction, and its structure changes depending on the type of problem. In binary classification problems, we usually use one single neuron with sigmoid activation, which gives a probability between 0 and 1. In multi-class classification problems, we use several neurons — one for each class — with softmax activation, which creates a probability for each class.

2.3 Mathematical Operation and Learning Process (Learning Mechanics)

Forward Propagation

Forward propagation is the process where input data flows from the input layer, moves through the hidden layers one by one, and finally reaches the output layer. At each layer, the network calculates a weighted sum and applies an activation function. Step by step, this builds up a prediction.

Loss Function

We use a loss function to measure how far the model's prediction is from the actual value. In regression problems, we usually use Mean Squared Error (MSE). In classification problems, we usually use Cross-Entropy loss.

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$$

$$Cross-Entropy = - \sum y_i \log(\hat{y}_i)$$

Backpropagation

Backpropagation is a powerful algorithm. It uses the chain rule from calculus to calculate the gradient — the partial derivative — of the loss function with respect to every single weight and bias. It sends the error

signal backward, starting at the output layer and moving toward the input layer, so that we can measure how much each parameter affects the loss.

Gradient Descent

Using the gradients found through backpropagation, the gradient descent algorithm updates each weight and bias by a small amount, in the direction opposite to the loss — that is, in the negative direction of the gradient. The size of this update is controlled by the learning rate. By repeating this process again and again, the loss function gradually moves toward its lowest value.

$$w := w - \alpha \cdot (\partial L / \partial w)$$

Here, α is the learning rate, which controls the size of each update step. In practice, instead of using the whole dataset at once, we calculate the gradient using small batches of data (mini-batches). This method is called Stochastic Gradient Descent (SGD), and it has improved versions such as Adam and RMSprop. This makes the training process faster and more efficient.

2.4 Theoretical Benchmark: Universal Approximation Theorem

The Universal Approximation Theorem is one of the most important foundations in the theory of neural networks. According to this theorem, a feedforward neural network with just one hidden layer — if it has enough neurons and a suitable non-linear activation function — can, in theory, approximate any continuous function to any level of accuracy we want.

However, this theorem has one important limitation: it only guarantees that such a network exists, but it does not tell us how to actually train that network well. In practice, instead of using a very wide but shallow network, using a deep network with several hidden layers can reach the same level of performance with far fewer parameters and more effective training. This is the practical reason why deep architectures are used.

3. Paradigm Comparison and Contrast Analysis

3.1 Traditional Machine Learning vs Deep Learning

The table below explains, on several points, the basic differences between traditional Machine Learning (ML) and Deep Learning (DL):

Point of Comparison	Traditional Machine Learning (ML)	Deep Learning (DL)
Data Dependency	Performs well even with small to medium amounts of data.	Needs a large amount of data for the best performance.

Point of Comparison	Traditional Machine Learning (ML)	Deep Learning (DL)
Hardware Needs	Can be trained quickly and efficiently on an ordinary CPU.	Parallel processing hardware such as a GPU or TPU is almost always needed.
Feature Extraction	Manual feature engineering is required; needs a domain expert.	Automatic representation learning; features are discovered on their own.
Training and Prediction Time	Training finishes quickly; prediction is also relatively fast.	Training can take from hours to weeks; prediction can be relatively slow.
Interpretability	Highly interpretable (for example, decision trees and linear regression are easy to explain).	"Black box" in nature — it is hard to explain the internal logic behind a decision.
Problem-Solving Approach	Pipeline-based — the problem is solved through several separate steps.	End-to-end learning — output comes directly from raw input.

3.2 Statistical Paradigm vs Logical/Neural Paradigm

Most traditional machine learning algorithms — such as Logistic Regression, Support Vector Machine (SVM), and Naive Bayes — are mainly built on statistical distribution and probability theory. These models reach their decisions based on the statistical structure hidden in the data, such as mean, variance, and conditional probability.

Deep learning, on the other hand, follows a hierarchical logical structure, inspired by how biological neurons connect with each other in the human brain. Here, knowledge is not compressed into a single statistical formula. Instead, it is stored as a distributed representation across millions of parameters, and complex patterns are identified through layer-by-layer logical abstraction. This is why deep learning is seen not simply as a statistical technique, but as a connectionist, brain-inspired computational paradigm.

3.3 Hardware Accelerators: CPU vs GPU vs TPU vs NPU

Feature	CPU	GPU	TPU (Tensor Processing Unit)	NPU / Edge TPU
Architecture	General-purpose; complex, logic-heavy cores.	A large collection of thousands of small, simple cores.	Custom ASIC built by Google; specialized for matrix multiplication.	Extremely small, power-saving chip for edge devices.

Feature	CPU	GPU	TPU (Tensor Processing Unit)	NPU / Edge TPU
Number of Cores	Usually 4 to 64 powerful cores.	Thousands (from a few thousand to 10,000+) of cores.	A large number of MXU units, based on a systolic array.	A limited number of specialized neural processing cores.
Type of Processing	Efficient at sequential processing.	Extremely efficient at parallel processing.	Specialized for tensor matrix multiplication (Tensor MXU).	Low-power, parallel processing focused on inference.
Power Efficiency	Medium; suitable for general tasks.	High power use, especially during training.	High performance-per-watt for both training and inference.	Very power-efficient; suitable for mobile and IoT.
Common Use	General computing, pre-processing, small models.	Model training in research labs and industry; graphics rendering.	Large-scale model training and inference on Google Cloud.	Smartphones, IoT devices, real-time edge inference.

4. The Ecosystem of Modern Deep Learning Architectures

Based on the direction that information flows, a neural network can be divided into two basic types: the feedforward network, where information flows only in one direction — from input to output — with no cycle; and the feedback, or recurrent, network, where the output or hidden state comes back again as input to the network, so the network becomes able to hold a kind of "memory." The specialized architectures described below have all developed on the basis of this basic difference.

4.1 ANN / MLP (Artificial Neural Network / Multi-Layer Perceptron)

ANN, or MLP, is the most basic feedforward architecture, where every neuron in a layer is connected to every neuron in the next layer (fully connected, or dense). It is mainly used to process tabular or structured data, such as numeric and categorical data taken from spreadsheets or databases.

4.2 CNN (Convolutional Neural Networks)

The Convolutional Neural Network is specialized for grid-shaped data — spatial grid data — especially for image processing. It uses the idea of a local receptive field, where each neuron looks at a small, specific region instead of the whole input. A convolution filter, or kernel, slides over the image to create a feature

map, which captures visual features such as edges, textures, and shapes. Through parameter sharing and pooling layers, a CNN stays computationally efficient while still achieving a high level of visual perception.

4.3 RNN / LSTM (Recurrent Neural Networks)

The Recurrent Neural Network is designed to process sequential or time-dependent data, such as text, speech, or stock prices. Its main feature is an internal loop, through which the hidden state from the previous time-step affects the calculation at the current time-step — this way, the network holds a kind of short-term memory. However, a plain RNN suffers from the vanishing gradient problem over long sequences. To solve this, architectures such as Long Short-Term Memory (LSTM) and the Gated Recurrent Unit (GRU) were introduced, where a system of gates makes it possible to preserve long-term dependencies.

4.4 Transformers

The Transformer architecture removes the limitation of sequential processing found in RNNs, by using the self-attention mechanism to process an entire sequence all at once, in parallel. Self-attention lets every word, or token, decide its importance based on its relationship with every other token in the sequence — this makes it possible to capture long-range dependencies very effectively. Introduced in the 2017 paper "Attention Is All You Need," this architecture is the foundation of modern Large Language Models (LLMs), such as the GPT and BERT series.

4.5 GAN (Generative Adversarial Networks)

The Generative Adversarial Network is built on a game-theory-based structure, where two networks — the Generator and the Discriminator — are trained against each other, in competition. The Generator tries to create synthetic (fake) data, while the Discriminator tries to tell real data apart from fake data. Through this competitive training process, the Generator gradually learns to create data realistic enough to fool even the Discriminator.

4.6 Diffusion Models

The Diffusion Model is a generative method inspired by the principles of thermodynamics, and it works in two stages. In the forward diffusion stage, random noise is added to a real image step by step, until it becomes completely disordered data. Then, in the reverse diffusion stage, the model learns to remove this noise step by step, until a coherent, realistic image is rebuilt. This method is currently the main foundation behind advanced text-to-image generation systems, such as Stable Diffusion and DALL·E.

5. The Revival of Deep Learning in the 2010s: Five Driving Forces

Although the theoretical foundation of neural networks was laid in the middle of the twentieth century, this field went through several "AI Winters" — periods of decline — from the 1960s to the 1990s, when research funding, interest, and progress almost came to a standstill, mainly due to a lack of sufficient data, computing power, and effective training algorithms. In the 2010s, deep learning experienced a true "renaissance," driven by the combined effect of five parallel forces.

5.1 The Dataset Revolution

The explosion of big data, caused by the spread of the internet and digitization, along with crowdsourcing platforms such as Amazon Mechanical Turk (MTurk), made it possible to collect huge amounts of labeled data. The ImageNet dataset, created in 2009 under the leadership of Fei-Fei Li, and the yearly competition built around it — the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) — became the center of this revolution. In addition, domain-specific open datasets such as MNIST (handwritten digits), CIFAR (small image classification), COCO (object detection and segmentation), SQuAD (question answering), Common Crawl (web text), and LibriSpeech (speech recognition) helped speed up progress in their specific research areas.

5.2 Hardware and Parallel Computing

Along with the steady rise in transistor density described by Moore's Law, NVIDIA's CUDA (Compute Unified Device Architecture) platform, introduced in 2006–07, opened up the Graphics Processing Unit (GPU) for general-purpose parallel computing. In 2012, the AlexNet model, built by Alex Krizhevsky, achieved a surprisingly strong result at the ILSVRC competition by using a GPU — this is recognized as a landmark moment in the history of deep learning. Later on, Google's Tensor Processing Unit (TPU) and various Neural Processing Units (NPUs) brought even more specialized and efficient hardware solutions.

5.3 Rise of Software Frameworks and Libraries

Early frameworks such as Theano, Caffe, and Torch were the first step in making deep learning research more accessible. Later, Google's TensorFlow and Meta's PyTorch became the two main pillars of the industry. TensorFlow first used a static computational graph, but with Keras integration and version 2.0 and above, it became dynamic and more user-friendly. PyTorch, on the other hand, gained wide popularity among researchers from the very beginning, through its dynamic computational graph and Autograd. In addition, high-level libraries such as Hugging Face Transformers, Fast.ai, and JAX made model development even faster and easier.

Point of Comparison	TensorFlow (Google)	PyTorch (Meta)
Computational Graph	Mainly a static graph (define-and-run); Eager Execution was added in version 2.0 and above.	A dynamic graph (define-by-run), which can be flexibly changed at runtime.
Learning Curve	Relatively easy, with a high-level API, due to Keras integration.	Pythonic and intuitive syntax, popular among researchers.
Production Deployment	A strong production ecosystem, including TensorFlow Serving, TensorFlow Lite, and TensorFlow.js.	Deployment through TorchServe and ONNX; steadily becoming stronger.
Main User Base	Industry and large-scale production systems.	Influential in academia and the research community; now also widely adopted in industry.

5.4 Algorithmic and Architectural Breakthrough Innovations

The biggest obstacle in training deep networks was the vanishing gradient problem, where, during backpropagation, the gradient value became so small at each layer that the weights in the early layers were not updated effectively. Introducing the ReLU activation function gave an effective solution to this problem, since it does not suffer from the same saturation issue as sigmoid. In addition, Batch Normalization stabilizes the input distribution at each layer, making training faster and more stable; Dropout prevents overfitting; and the skip connections introduced in ResNet make it possible to train extremely deep networks with hundreds of layers. On top of all this, the self-attention process, through the Transformer architecture, brought a revolutionary change to sequence modeling.

5.5 Open Science and Global Community (Open Science)

Pre-print platforms such as arXiv created an opportunity to publish research results quickly and openly, skipping the long delays of the traditional peer-review process. The culture of sharing code openly through GitHub ensures the reproducibility of research. The Hugging Face model hub makes trained models easily available, and competitive platforms such as Kaggle encourage knowledge exchange and innovative competition among researchers and practitioners around the world.

6. Special Tips and Insights

Insight: Will Deep Learning Completely Replace Traditional Machine Learning?

No — instead, the idea of "the right tool for the right job" applies here. For tabular or structured data, gradient boosted tree algorithms such as XGBoost or LightGBM often give faster results, use fewer computing resources, and are more accurate than deep neural networks.

In cases where the amount of data is limited (data scarcity), or the computing budget is small, a traditional ML model is often a more practical and cost-effective solution.

In the legal and financial sectors, where interpretability is a regulatory requirement, the transparency of a decision tree or a linear model is more acceptable than the black-box nature of deep learning.

Insight: The Black Box Problem

Because a deep neural network has millions of parameters and many non-linear transformations, it is very hard to explain the exact logic behind a specific decision — this lack of clarity is called the "black box" problem.

In healthcare, if a doctor or a patient cannot understand the reason behind a diagnosis decision, it can create ethical and legal complications. Similarly, in finance, a lack of interpretability in loan approval or rejection decisions increases the risk of discriminatory outcomes.

To solve this problem, Explainable AI (XAI) techniques — such as SHAP and LIME — are being used to make a model's decisions easier for people to understand.

7. Chapter Review and Assessment

7.1 Summary

- Deep learning is a representation-learning method based on multi-layer neural networks, which automatically learns layered features from raw data.
- Starting from the perceptron, a neural network is trained through the combination of forward propagation, the loss function, backpropagation, and gradient descent.
- The Universal Approximation Theorem establishes the theoretical capability of neural networks, but a deep architecture is needed for practical trainability.
- There are clear differences between traditional ML and DL in terms of data dependency, hardware requirements, feature extraction, and interpretability.
- CNN, RNN/LSTM, Transformers, GAN, and Diffusion Models — each architecture is specialized for a particular type of data and problem.
- The revival of the 2010s is the combined result of datasets, hardware, software frameworks, algorithmic innovation, and the open science community.

- Deep learning does not replace traditional machine learning — rather, both complement each other in specific situations.

7.2 Key Mathematical Formulas

Perceptron

$$z = w^T x + b$$

$$a = f(z)$$

MSE Loss

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Cross-Entropy Loss

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

Gradient Descent Update

$$w := w - \alpha \frac{\partial L}{\partial w}$$

Softmax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

7.3 Top 5 Interview Questions and Formal Answers

Question 1: What is representation learning, and how is it different from manual feature engineering?

Answer: Representation learning is a process where a model automatically discovers useful features from raw data, without any human help. In contrast, manual feature engineering requires a domain expert to identify important features by hand, which can be time-consuming and can carry bias. A deep learning model overcomes this limitation by learning increasingly abstract representations, layer by layer.

Question 2: What is the vanishing gradient problem, and how did ReLU/ResNet solve it?

Answer: The vanishing gradient problem happens when, during backpropagation, the gradient value becomes exponentially smaller with each layer it passes through, so the early layers do not learn effectively. The ReLU activation function reduces this problem by avoiding the saturation issue seen with sigmoid. The skip connections introduced in ResNet give the gradient a direct alternative path back to earlier layers, making it possible to train extremely deep networks effectively.

Question 3: Compare the architectures of CPU, GPU, TPU, and NPU in the context of tensor operations.

Answer: A CPU is efficient at sequential tasks with a limited number of powerful cores, but it is inefficient at large matrix computations. A GPU provides massive parallelism through thousands of small cores, which is well suited to the matrix multiplication used in neural networks. A TPU is a custom-built ASIC

from Google, specialized for tensor multiplication through a systolic array design, and it offers high performance-per-watt for both training and inference. An NPU, or Edge TPU, is an extremely small, power-saving chip designed for real-time inference on mobile and IoT devices.

Question 4: Why are gradient boosted trees, such as XGBoost, often preferred over deep neural networks for tabular data?

Answer: In tabular data, the relationships between features are often discrete and non-spatial, which is exactly where decision-tree-based methods are naturally efficient. Models like XGBoost often give equal or better results than deep neural networks, even with less training data, fewer computing resources, and less hyperparameter tuning — and on top of that, they are more interpretable, through feature importance.

Question 5: How does the Transformer's self-attention mechanism overcome the limitations of RNNs?

Answer: An RNN processes tokens one at a time, sequentially, which makes parallelization difficult and causes it to fail at capturing long-range dependencies in long sequences. The self-attention mechanism processes all tokens at once and directly calculates the relationship weight between every pair of tokens, so the relationship between any two tokens can be captured equally well regardless of distance — and the training process becomes highly parallelizable on a GPU.

7.4 Multiple Choice Questions (MCQs)

1. In the term "Deep Learning," what does the word "Deep" mainly refer to?

- (a) The size of the training data
- (b) The number of hidden layers in the network
- (c) The number of GPUs used
- (d) The value of the learning rate

Correct answer: (b) The number of hidden layers in the network.

2. What is the name of the model that achieved a breakthrough result at the ILSVRC competition in 2012, using a GPU?

- (a) ResNet
- (b) LeNet
- (c) AlexNet
- (d) VGGNet

Correct answer: (c) AlexNet.

3. Which of the following directly helps reduce the vanishing gradient problem?

- (a) Softmax activation
- (b) Skip connections (ResNet)
- (c) One-hot encoding
- (d) Min-max scaling

Correct answer: (b) Skip connections (ResNet).

4. Which of the following architectures is built on the self-attention mechanism?

- (a) LSTM
- (b) CNN
- (c) Transformer
- (d) GAN

Correct answer: (c) Transformer.

5. What is the name of the custom hardware chip, built by Google, that is specialized for tensor multiplication?

- (a) NPU
- (b) TPU
- (c) FPGA
- (d) ASIC (general)

Correct answer: (b) TPU (Tensor Processing Unit).

7.5 Analytical Questions and Practical Problems

Problem 1

A bank wants to build a loan-approval decision system, where the law requires that the reason behind every decision must be clearly explainable to the regulator. Would you recommend a deep neural network, or a traditional machine learning model (such as logistic regression or a decision tree) in this case? Present your argument, considering interpretability, the amount of data available, and the regulatory requirement.

Problem 2

A company that builds autonomous vehicles wants to process, at the same time, images of the road (visual data) and the driver's voice commands (sequential data). Which deep learning architectures do you think would be suitable for these two types of data, and why? Explain how the structural features of each architecture match the nature of its related data.